

1. SQL Query Execution Order (VERY IMPORTANT)

Even though we *write* SQL in one order, it *executes* in another:

ACTUAL EXECUTION ORDER

FROM
JOIN
WHERE
GROUP BY
HAVING
SELECT
DISTINCT
ORDER BY
LIMIT

Example

```
SELECT dept_id, COUNT(*)  
FROM employees  
WHERE salary > 50000  
GROUP BY dept_id  
HAVING COUNT(*) > 2  
ORDER BY dept_id;
```

 Execution:

1. FROM employees
 2. WHERE salary > 50000
 3. GROUP BY dept_id
 4. HAVING COUNT(*) > 2
 5. SELECT dept_id, COUNT(*)
 6. ORDER BY dept_id
-

2. General SELECT Syntax (Write Order)

```
SELECT column1, aggregate_function(column2)
FROM table_name
[JOIN table2 ON condition]
WHERE condition
GROUP BY column1
HAVING condition
ORDER BY column1
LIMIT n;
```

3. RULES YOU MUST REMEMBER (INTERVIEW GOLD)

◆ SELECT + GROUP BY Rule

Every non-aggregated column in SELECT must be in GROUP BY

```
-- ✗ WRONG
SELECT dept_id, salary, COUNT(*)
FROM employees
GROUP BY dept_id;
```

```
-- ✓ CORRECT
SELECT dept_id, COUNT(*)
FROM employees
GROUP BY dept_id;
```

◆ WHERE vs HAVING

Clause	Use
WHERE	Filters rows BEFORE grouping
HAVING	Filters AFTER grouping

```
-- ✗ WRONG
WHERE COUNT(*) > 2
```

```
-- ✓ CORRECT
HAVING COUNT(*) > 2
```

◆ Can HAVING exist without GROUP BY?

👉 YES (interview trap)

```
SELECT COUNT(*)  
FROM employees  
HAVING COUNT(*) > 5;
```

✓ Treats whole table as one group

◆ ORDER of GROUP BY and HAVING

GROUP BY → HAVING

👉 HAVING always comes **after** GROUP BY

◆ DISTINCT position

```
SELECT DISTINCT column1  
FROM table;
```

👉 Comes **right after** SELECT

◆ Aggregate functions and NULL

Function	NULL handling
COUNT(*)	counts all rows
COUNT(col)	ignores NULL
SUM/AVG	ignore NULL
MIN/MAX	ignore NULL

◆ Aliases usage

```
SELECT salary AS sal  
FROM employees;
```

👉 Cannot use alias in WHERE ❌

👉 Can use in ORDER BY ✅

◆ LIMIT

```
SELECT *  
FROM employees  
ORDER BY salary DESC  
LIMIT 5;
```

👉 Always comes at the end

🔗 4. JOIN Syntax + Rules

```
SELECT columns  
FROM table1  
JOIN table2  
ON table1.col = table2.col;
```

🔥 Types of JOIN

- INNER JOIN
 - LEFT JOIN
 - RIGHT JOIN
 - FULL JOIN (not in MySQL directly)
-

⚠️ Rules

- JOIN happens **before** WHERE
 - Always use **ON** condition
 - Avoid ambiguous column names → use aliases
-

✎ 5. DML (Data Manipulation Language)

◆ INSERT

```
INSERT INTO table_name (col1, col2)
VALUES (val1, val2);
```

◆ UPDATE

```
UPDATE table_name
SET column1 = value1
WHERE condition;
```

⚠ Without WHERE → updates ALL rows

◆ DELETE

```
DELETE FROM table_name
WHERE condition;
```

⚠ Without WHERE → deletes ALL rows



6. DDL (Data Definition Language)

◆ CREATE

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype
);
```

◆ ALTER

```
ALTER TABLE table_name
ADD column_name datatype;
```

◆ DROP

DROP TABLE table_name;

◆ TRUNCATE

TRUNCATE TABLE table_name;

⚠ TRUNCATE vs DELETE

Feature	DELETE	TRUNCATE
WHERE allowed	✓	✗
Faster	✗	✓
Resets auto increment	✗	✓

🧠 7. Constraints (DDL rules)

```
CREATE TABLE employees (  
  emp_id INT PRIMARY KEY,  
  name VARCHAR(50) NOT NULL,  
  salary INT,  
  dept_id INT,  
  UNIQUE(name),  
  CHECK (salary > 0)  
);
```

🔥 8. Subqueries Rules

◆ Types

- Scalar (single value)
- Correlated
- Nested

◆ Rules

- Subquery inside WHERE must return compatible value
 - Correlated subquery references outer query
 - **IN, EXISTS, ANY, ALL** used often
-

⚡ 9. Window Functions (Advanced but important)

```
SELECT emp_id,  
       AVG(salary) OVER (PARTITION BY dept_id)  
FROM employees;
```

Rules:

- Do NOT collapse rows (unlike GROUP BY)
 - Work after SELECT stage
-

🚨 10. Common Interview Mistakes

- ✗ Using aggregate in WHERE
 - ✗ Forgetting GROUP BY column
 - ✗ Misunderstanding NULL
 - ✗ Confusing WHERE vs HAVING
 - ✗ Wrong JOIN conditions
 - ✗ Using alias in WHERE
-

🧠 Final Mental Model

Think SQL like this:

1. Get data (FROM + JOIN)

2. Filter rows (WHERE)
 3. Group data (GROUP BY)
 4. Filter groups (HAVING)
 5. Select columns (SELECT)
 6. Sort (ORDER BY)
 7. Limit (LIMIT)
-

ORDER BY INFO-



arrange in alphabetical order sql

Reverse Alphabetical Sorting (Z to A)

To sort in descending order, use the `DESC` keyword after the column name. [W3Schools +1](#)

sql



```
SELECT * FROM table_name  
ORDER BY column_name DESC;
```

Sorting by Multiple Columns

If multiple rows have the same value in the first column, you can provide a secondary column to determine the [alphabetical order for those rows](#). [@](#)

sql



```
SELECT * FROM Employees  
ORDER BY last_name ASC, first_name ASC;
```


Database 

id	name	Country	Score
1	Maria	Germany	350
2	John	USA	900
3	Georg	UK	750
4	Martin	Germany	500
5	Peter	USA	0

Lowest ↑

4	Martin	Germany	500
1	Maria	Germany	350

↑ Highest

↓ Highest

3	Georg	UK	750
2	John	USA	900
5	Peter	USA	0

↓ Lowest

③ SELECT *

① FROM Table

② ORDER BY

Country ASC,

Score DESC



SQL SELECT Queries (Visually Explained) for Beginners | All Essential Clauses | #SQL Course 4

GROUP BY VISUAL-

Database 

id	name	Country	Score
1	Maria	Germany	350
2	John	USA	900
3	Georg	UK	750
4	Martin	Germany	500
5	Peter	USA	0

1	Maria	Germany	350
2	John	USA	900
3	Georg	UK	750
4	Martin	Germany	500
5	Peter	USA	0

Germany	350
Germany	500

SELECT

Country,

SUM(score)

① FROM Table

② GROUP BY Country




SQL SELECT Queries (Visually Explained) for Beginners | All Essential Clauses | #SQL Course 4

Database

id	name	Country	Score
1	Maria	Germany	350
2	John	USA	900
3	Georg	UK	750
4	Martin	Germany	500
5	Peter	USA	0

id	name	Country	Score
1	Maria	Germany	350
2	John	USA	900
3	Georg	UK	750
4	Martin	Germany	500
5	Peter	USA	0

Germany	850
USA	900
UK	750

SELECT

Country,
SUM(score)

① FROM Table

② GROUP BY Country

SQL SELECT Queries (Visually Explained) for Beginners | All Essential Clauses | [#SQL](#)
Course 4

But what if i want to display other columns also which i dont want to be in group by

Problem Recap

You want:

```
SELECT dept_id, salary, COUNT(*)
FROM employees
GROUP BY dept_id;
```

👉 But SQL says ✗ ERROR

WHY SQL doesn't allow this

For a single dept_id, there are **multiple salaries**:

Example (dept 101):

50000, 70000, 70000

👉 So SQL asks:

“Which salary should I show?? 🤔”

- 50000?
- 70000?
- Which row?

👉 **Ambiguous** → **SQL rejects it**

✅ So how to solve this?

You have **3 valid approaches**, depending on what you actually want.

✅ Option 1: Aggregate the column

If you want *one value per group*, aggregate it:

```
SELECT dept_id, MAX(salary), COUNT(*)  
FROM employees  
GROUP BY dept_id;
```

👉 Now it makes sense:

- One dept → one max salary

✅ Option 2: Use **ANY_VALUE()** (MySQL-specific ⚠️)

```
SELECT dept_id, ANY_VALUE(salary), COUNT(*)  
FROM employees  
GROUP BY dept_id;
```

👉 Meaning:

“I don’t care which salary, give me any”

⚠ Interview note:

- Works in MySQL
- But **not recommended in real logic questions**

✅ Option 3: Use Subquery / JOIN (BEST for interviews)

If you want:

“Show full row corresponding to aggregated value”

Example: employee with max salary per dept

```
SELECT *  
FROM employees e1  
WHERE salary = (  
    SELECT MAX(e2.salary)  
    FROM employees e2  
    WHERE e2.dept_id = e1.dept_id  
);
```

✅ Option 4: Window Functions (BEST modern solution 🔥)

```
SELECT emp_id, dept_id, salary,  
    MAX(salary) OVER (PARTITION BY dept_id) AS max_salary  
FROM employees;
```

👉 You get:

- All rows
- Plus aggregate info

🔥 Interview-Level Insight

! Golden Rule

If a column is not aggregated, it must be functionally dependent on GROUP BY

! Special Case (Advanced)

If table has:

dept_id → dept_name (1-to-1 mapping)

Then this works:

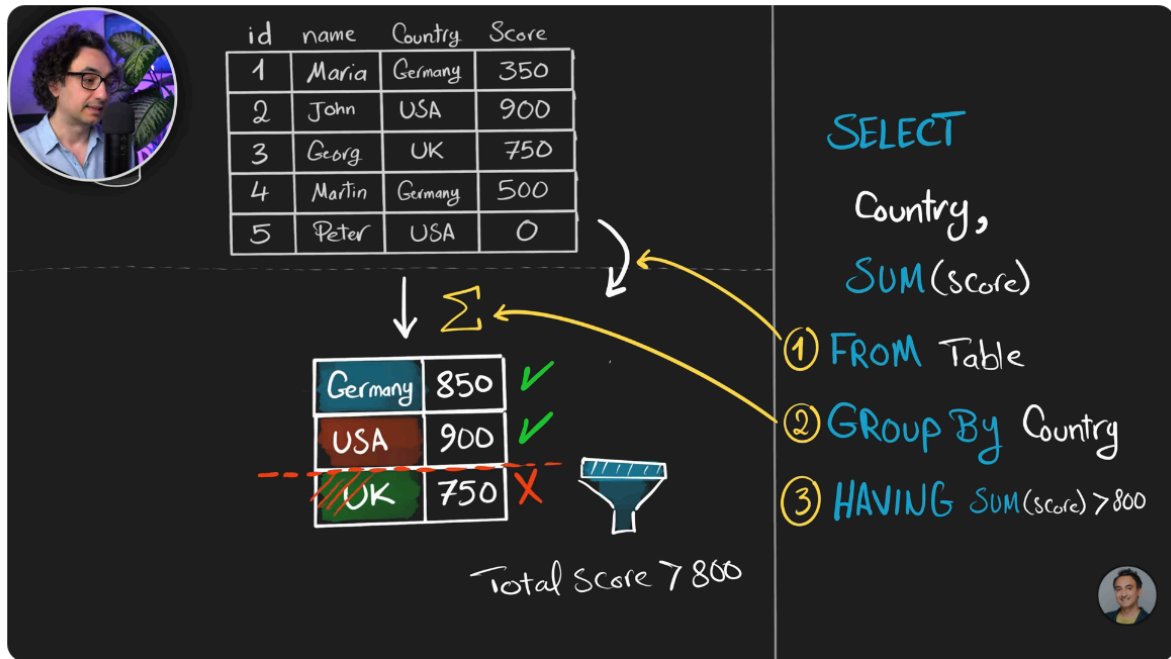
```
SELECT dept_id, dept_name  
FROM dept_table  
GROUP BY dept_id;
```

👉 Because:

- dept_name is uniquely determined by dept_id

🚀 Summary

Goal	Solution
Show summary per group	GROUP BY + aggregates
Show random value	ANY_VALUE()
Show full row of max/min	Subquery / JOIN
Show row + aggregates	Window functions



SQL SELECT Queries (Visually Explained) for Beginners | All Essential Clauses | [#SQL](#)
Course 4

But why cant i use aggregate functions directly in where but i can in having clause

🧠 Short Answer (Interview Style)

You can't use aggregate functions in **WHERE** because **WHERE** is executed before aggregation happens.

You can use them in **HAVING** because **HAVING** is executed after aggregation.

🔥 Let's Break It Properly

📌 SQL Execution Order (again, very important)

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Why aggregates don't work in WHERE

Example:

```
SELECT dept_id  
FROM employees  
WHERE AVG(salary) > 50000  
GROUP BY dept_id;
```

What SQL sees:

Step 1: FROM → get all rows
Step 2: WHERE → filter rows

👉 But at this point:

- No grouping yet
- No AVG calculated yet

So SQL is like:

“AVG of what?? I haven't grouped anything yet 😞”

  ERROR

Why HAVING works

```
SELECT dept_id  
FROM employees  
GROUP BY dept_id  
HAVING AVG(salary) > 50000;
```

What happens:

1. FROM → get rows

2. WHERE → (if any filter)
3. GROUP BY → create groups
4. AVG(salary) → computed per group
5. HAVING → filter groups

👉 Now AVG exists → ✅ valid

Visual Understanding

Imagine this:

Table:

dept_id	salary
---------	--------

101	50000
-----	-------

101	70000
-----	-------

102	60000
-----	-------

WHERE stage:

Row level data (no grouping yet)

GROUP BY stage:

Dept 101 → [50000, 70000]

Dept 102 → [60000]

HAVING stage:

Now AVG exists:

Dept 101 → 60000

Dept 102 → 60000

👉 Now filtering works

Key Difference

Clause	Works on	Can use aggregates?
WHERE	Individual rows	❌ No
HAVING	Groups of rows	✅ Yes

Common Interview Trap

❌ Wrong

WHERE COUNT(*) > 2

✅ Correct

HAVING COUNT(*) > 2

Advanced Insight

You *can* indirectly use aggregates in WHERE using a **subquery**:

```
SELECT *  
  
FROM employees  
  
WHERE salary > (  
  
    SELECT AVG(salary) FROM employees  
  
);
```

👉 Why this works:

- Subquery runs separately → gives a value
- WHERE just compares values → no aggregation happening there

Final Mental Model

Think:

WHERE → filter raw rows

GROUP BY → create groups

HAVING → filter grouped results

Correct Mental Model

👉 It's not that SQL "can't see columns"

👉 It's that:

Aggregates are defined on a *set of rows*, not on a single row

Why your assumption felt logical

You thought:

WHERE AVG(salary) > 50000

👉 might mean:

“Take average of entire salary column”

That *would* make sense logically...

But SQL doesn't work like that

At the **WHERE stage**, SQL is processing **one row at a time**

Think:

Row 1 → salary = 50000 → should I keep it?

Row 2 → salary = 70000 → should I keep it?

👉 WHERE expects a condition like:

salary > 50000

✓ Something that can be evaluated **per row**

Why AVG fails here

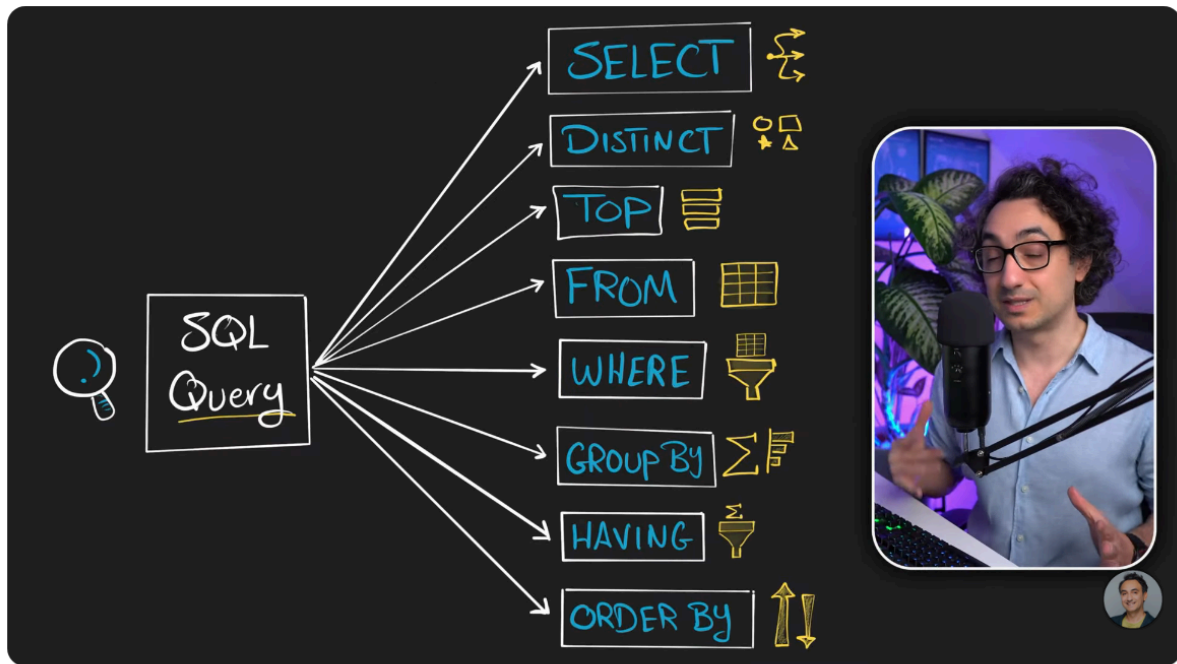
AVG(salary)

👉 Needs:

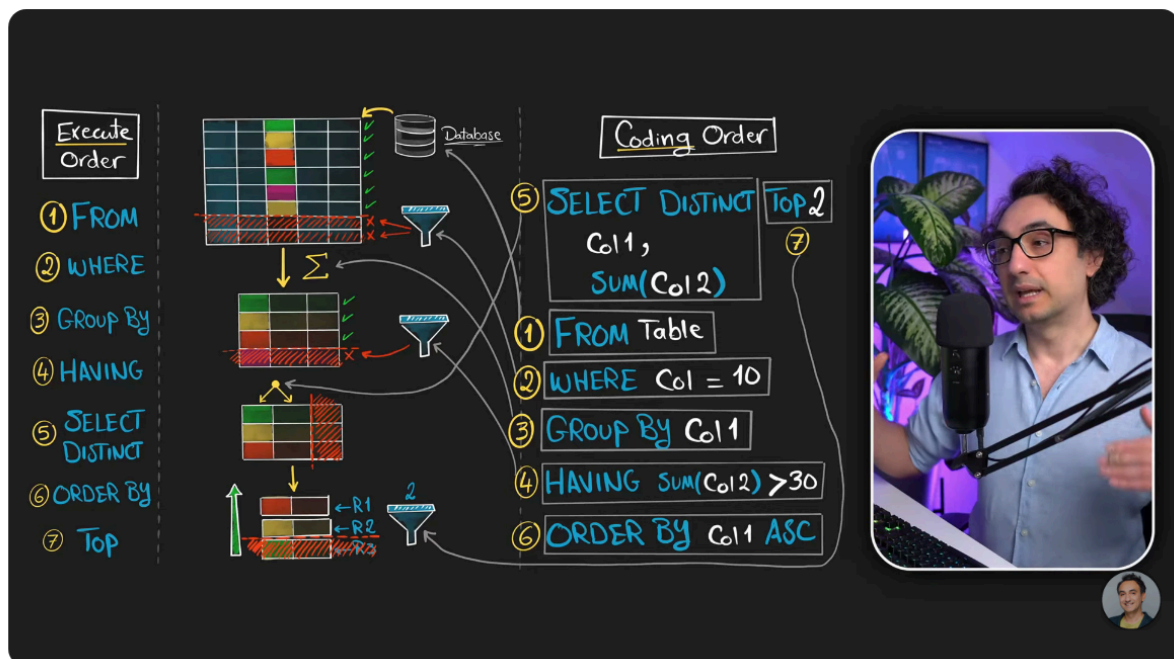
- multiple rows
- grouping or full dataset

But WHERE is like:

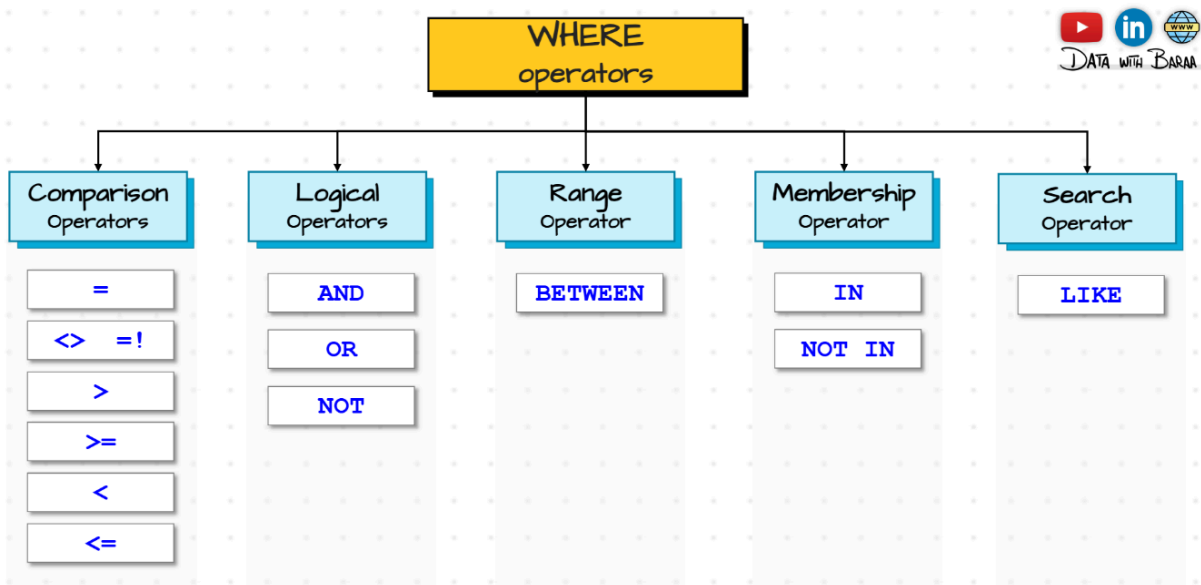
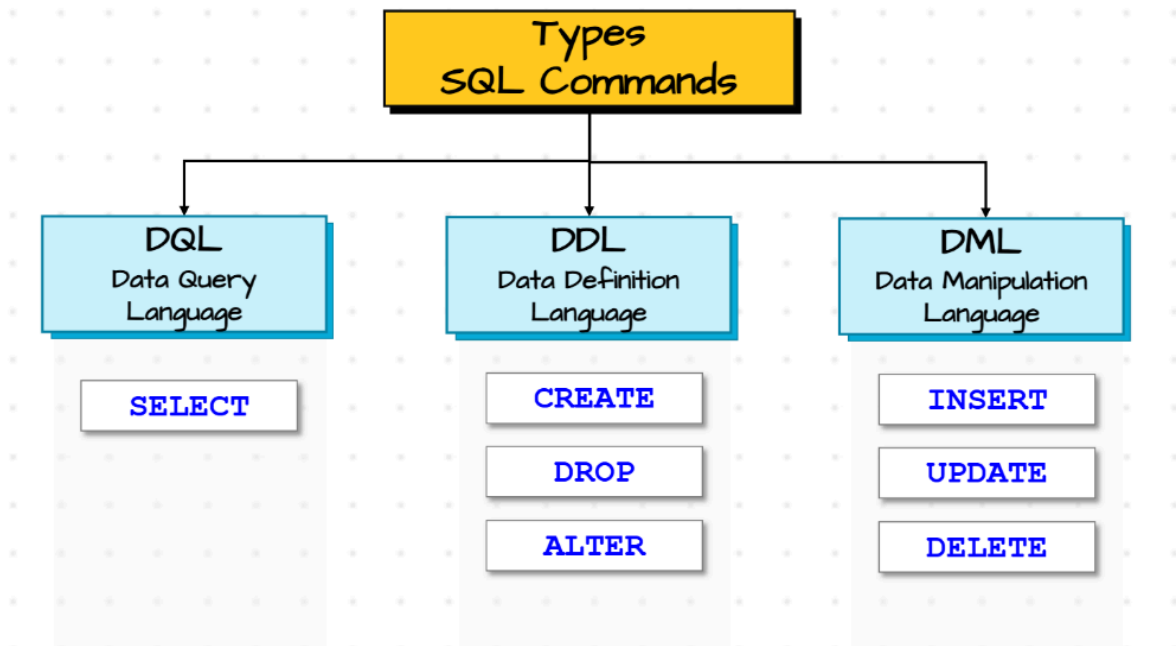
“I’m checking ONE row... how do I compute an average?? 🤔”

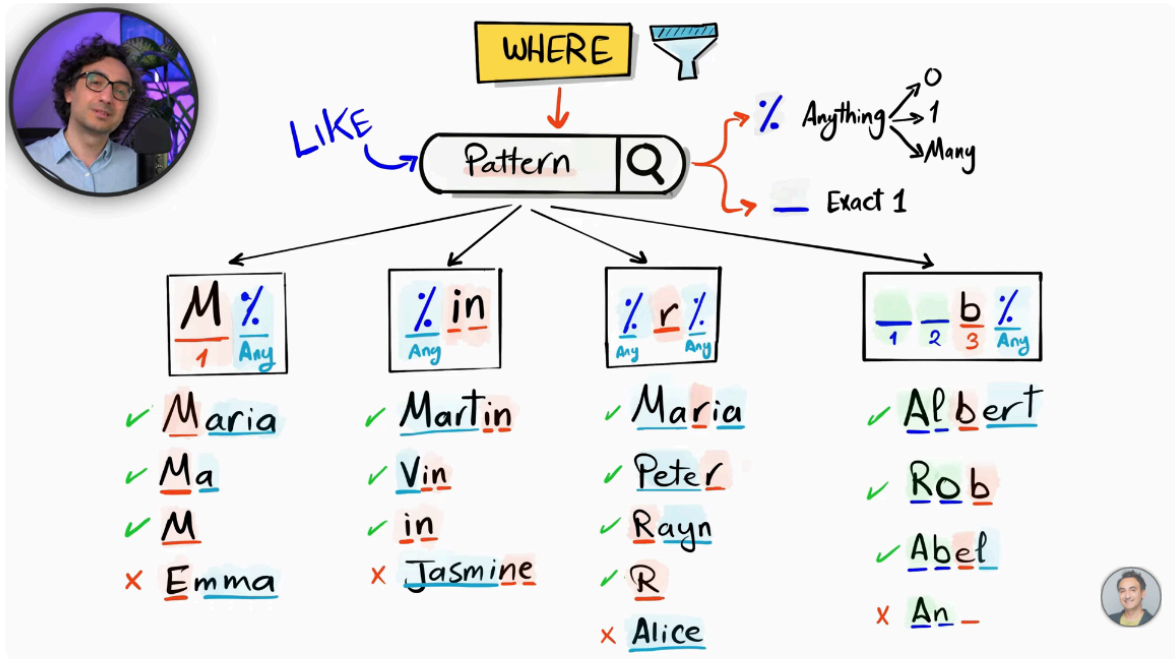


SQL SELECT Queries (Visually Explained) for Beginners | All Essential Clauses | #SQL Course 4



SQL SELECT Queries (Visually Explained) for Beginners | All Essential Clauses | #SQL Course 4





SQL WHERE Conditions (Visually Explained) | AND, OR, NOT, LIKE, BETWEEN, IN |
[#SQL Course 7](#)

CASE STATEMENT-

“Can you create **derived / calculated columns** in SQL?”

🧠 What they mean

If interviewer says:

“Based on employee salary, create columns like X, Y, Z (like showing titles as fresher, mid, senior based on salary, YOE, etc)”

👉 They want you to use:

👉 CASE WHEN (MOST IMPORTANT 🔥)

✅ Basic Syntax

SELECT

column1,

```
CASE

  WHEN condition1 THEN value1

  WHEN condition2 THEN value2

  ELSE value3

END AS new_column

FROM table;
```

Example 1: Categorize salary

```
SELECT name, salary,

CASE

  WHEN salary < 50000 THEN 'Low'

  WHEN salary BETWEEN 50000 AND 80000 THEN 'Medium'

  ELSE 'High'

END AS salary_category

FROM employees;
```

Output

name	salary	salary_category
Alice	50000	Medium
Eve	90000	High

Example 2: Multiple calculated columns (X, Y, Z)

👉 Suppose they ask:

“Create 3 columns based on salary”

SELECT name, salary,

CASE WHEN salary > 60000 THEN 1 ELSE 0 END AS high_salary_flag,

CASE WHEN salary BETWEEN 50000 AND 70000 THEN 'Mid' ELSE 'Other' END AS mid_range,

CASE WHEN salary IS NULL THEN 'Missing' ELSE 'Present' END AS salary_status

FROM employees;

Key Idea

👉 Each column = separate **CASE**

👉 You can create **as many derived columns as you want**

Example 3: Conditional Aggregation

👉 This is **interview favorite**

“Count high salary vs low salary per department”

SELECT dept_id,

COUNT(CASE WHEN salary > 60000 THEN 1 END) AS high_salary_count,

COUNT(CASE WHEN salary <= 60000 THEN 1 END) AS low_salary_count

FROM employees

GROUP BY dept_id;

Example 4: SUM with CASE (cleaner)

SELECT dept_id,

SUM(CASE WHEN salary > 60000 THEN 1 ELSE 0 END) AS high_salary_count,

SUM(CASE WHEN salary <= 60000 THEN 1 ELSE 0 END) AS low_salary_count

FROM employees

GROUP BY dept_id;

Important Rules

♦ CASE works in:

- SELECT 
- WHERE 
- HAVING 
- ORDER BY 

♦ Always give alias

END AS column_name

- ◆ **NULL handling**

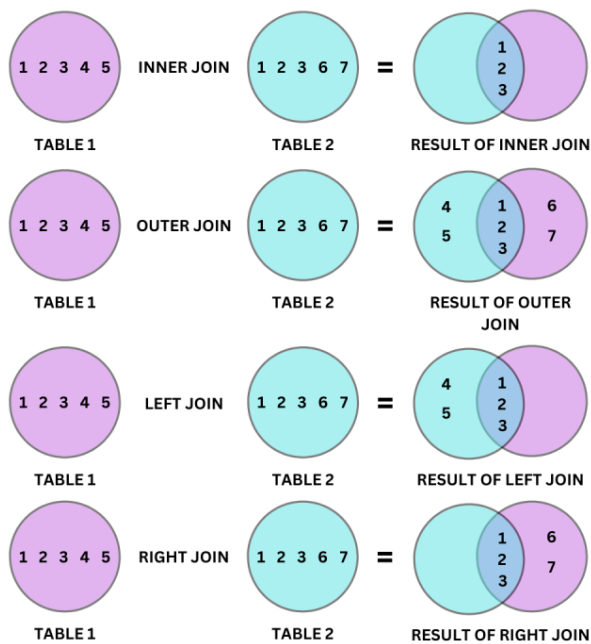
CASE

WHEN salary IS NULL THEN 'Missing'

Interview Pattern Recognition

If they say:

They say	You think
“categorize”	CASE
“create flag”	CASE
“separate columns”	CASE
“conditional count”	CASE + COUNT/SUM
“pivot-like output”	CASE



1. INTRODUCTION

1.1 Definition of SQL

1.2 Concept of SQL Joins

1.3 Need for SQL Joins

2. TYPES OF JOINS

2.1 Inner Join or Join

2.2 Full Outer Join or Outer Join

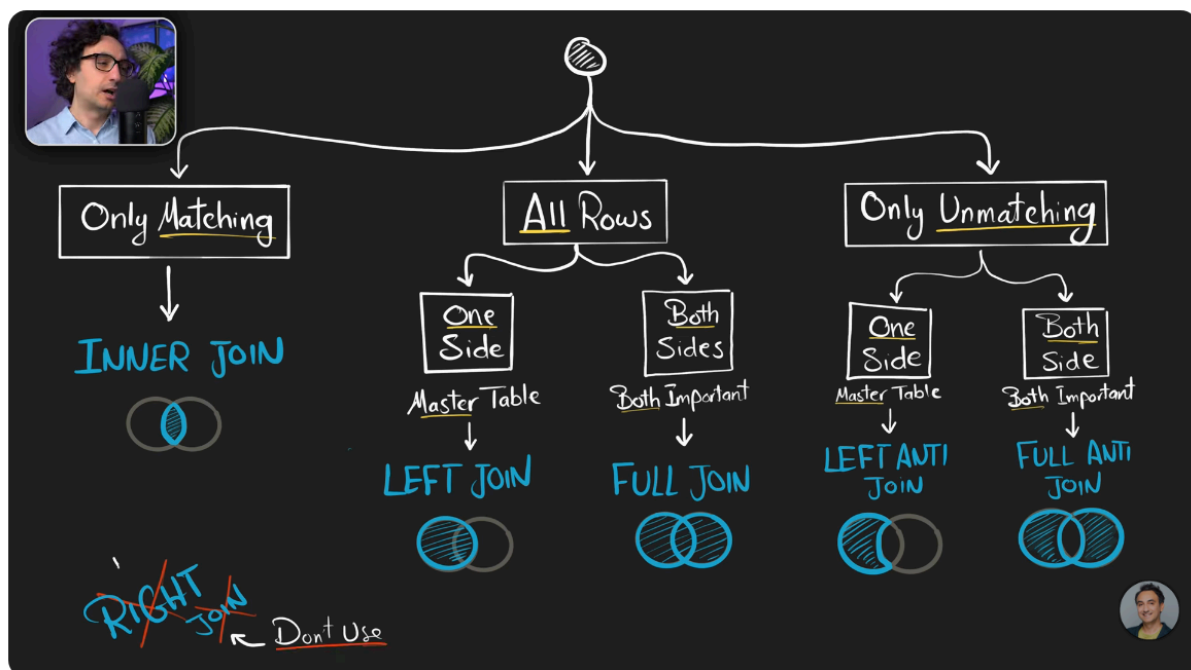
2.3 Left Outer Join or Left Join

2.4 Right Outer Join or Right Join

2.5 Self Join

2.6 Cross Join

3.0 REFERENCES



Joining Multiple Tables in SQL (Visually Explained) | 3+ Table Joins | #SQL Course 10

RIGHT JOIN is generally avoided in SQL development primarily for reasons of **readability, team consistency, and cognitive ease**, rather than technical limitations. Because most languages are read from left to right, **LEFT JOIN** is considered more intuitive—allowing the primary table to be listed first—while **RIGHT JOIN** forces developers to "invert their thinking" and read queries backwards.

Here is a detailed breakdown of why `RIGHT JOIN` is avoided:

- **Readability and Mental Effort:** `RIGHT JOIN` requires more mental effort to parse, especially in complex queries with multiple joins, as it forces the reader to track data from the bottom up or right-to-left.
- **Redundancy:** Every `RIGHT JOIN` can be rewritten as a `LEFT JOIN` by simply swapping the order of the tables. For instance, `TableA RIGHT JOIN TableB` is identical to `TableB LEFT JOIN TableA`.
- **Team Standardization:** To avoid confusion and make codebases easier to maintain, most teams adopt a convention of using only `LEFT JOIN`.
- **Harder Debugging:** When debugging, it is more straightforward to track which table is the "primary" one when using `LEFT JOIN`. `RIGHT JOIN` makes tracing data flow more difficult.
- **Confusing with Multiple Joins:** When mixing `LEFT` and `RIGHT` joins, queries can become highly ambiguous and difficult to debug, often leading to unintended results.
- **Unusual in Documentation:** Most tutorials, documentation, and best-practice guides teach and use `LEFT JOIN` as the standard, making `RIGHT JOIN` less familiar to developers.

When `RIGHT JOIN` is acceptable:

The only common, legitimate use case for a `RIGHT JOIN` is to avoid re-writing a long, complex, pre-existing query when a new table needs to be tacked onto the end, saving time on maintenance.



One-line Difference (Memorize This)

JOIN = combine columns (horizontal)

UNION = combine rows (vertical)



1. FULL JOIN (FULL OUTER JOIN)



What it does:

Combines **two tables side-by-side** based on a condition

```
SELECT *
```

```
FROM A
```

```
FULL JOIN B
```

```
ON A.id = B.id;
```



Example

Table A

id	name
----	------

1	Alice
---	-------

2	Bob
---	-----

Table B

id	salary
----	--------

2	70000
---	-------

3	90000
---	-------

Result (FULL JOIN)

id	name	salary
----	------	--------

1	Alice	NULL
---	-------	------

2	Bob	70000
---	-----	-------

3	NULL	90000
---	------	-------

Key Points

- Keeps **all rows from both tables**
- Matches where possible
- Fills NULL where no match

MySQL NOTE

 MySQL **does NOT** support FULL JOIN directly

You simulate it using:

```
SELECT * FROM A
```

```
LEFT JOIN B ON A.id = B.id
```

```
UNION
```

```
SELECT * FROM A
```

```
RIGHT JOIN B ON A.id = B.id;
```



2. UNION



What it does:

Combines **rows from two queries**

```
SELECT col1, col2 FROM A
```

```
UNION
```

```
SELECT col1, col2 FROM B;
```



Key Points

- Combines rows vertically
- Removes duplicates (by default)



UNION vs UNION ALL

UNION → removes duplicates

UNION ALL → keeps duplicates



IMPORTANT DIFFERENCES

Feature	FULL JOIN	UNION
Combines	Columns	Rows
Needs condition	✓ Yes	✗ No

Column count must match

✗ No

✓ Yes

Based on key match

✓ Yes

✗ No

Removes duplicates

✗ No

✓ (UNION only)

Visual Understanding

FULL JOIN

Table A → columns

Table B → columns

→ Merge side-by-side

UNION

Query 1 rows

Query 2 rows

→ Stack on top

Interview Trick Question

👉 Can you UNION tables with different column names?

✓ Yes — names don't matter

! But number + type must match



When to use what?

Situation	Use
Combine related data using key	JOIN
Combine similar datasets	UNION
Want all data even if no match	FULL JOIN
Merge results from multiple queries	UNION



Final Memory Trick

JOIN = widen table (→ columns)

UNION = lengthen table (↓ rows)



What is a Self Join?

A **self join** is when a table is joined with itself.

👉 Used when:

- Data in one row is related to another row in the same table



Classic Example: Employee–Manager



Table: **employees**

emp_id	name	manager_id
--------	------	------------

1	Alice	NULL
---	-------	------

2	Bob	1
---	-----	---

3	Charlie	1
---	---------	---

4	David	2
---	-------	---



Goal

👉 Show:

Employee name + their Manager name



Self Join Query

```
SELECT e.name AS employee, m.name AS manager
```

```
FROM employees e
```

```
LEFT JOIN employees m
```

```
ON e.manager_id = m.emp_id;
```



Explanation

- `employees e` → acts as employee table
- `employees m` → acts as manager table

👉 Same table, different roles



Output

<code>employee e</code>	<code>manage r</code>
-----------------------------	---------------------------

Alice	NULL
-------	------

Bob	Alice
-----	-------

Charlie	Alice
---------	-------

David	Bob
-------	-----



Why LEFT JOIN?

- So employees without manager (like Alice) are still shown



Important Rule

In self join, **aliases** are mandatory

`employees e`

employees m

👉 Otherwise SQL can't differentiate

Another Common Example

Find employees working in same department

```
SELECT e1.name, e2.name, e1.dept_id  
FROM employees e1  
JOIN employees e2  
ON e1.dept_id = e2.dept_id  
AND e1.emp_id != e2.emp_id;
```

💡 **Output meaning:**

- Pairs of employees in same dept

Another Interview Favorite

Find duplicate salaries

```
SELECT e1.emp_id, e1.salary  
FROM employees e1  
JOIN employees e2  
ON e1.salary = e2.salary  
AND e1.emp_id != e2.emp_id;
```



When to Think “Self Join”?

If question involves:

- “same table comparison”
- “hierarchy (manager, parent)”
- “pairs within same data”
- “duplicates”



Think **SELF JOIN**



Common Mistake

```
SELECT * FROM employees
```

```
JOIN employees ON ...
```

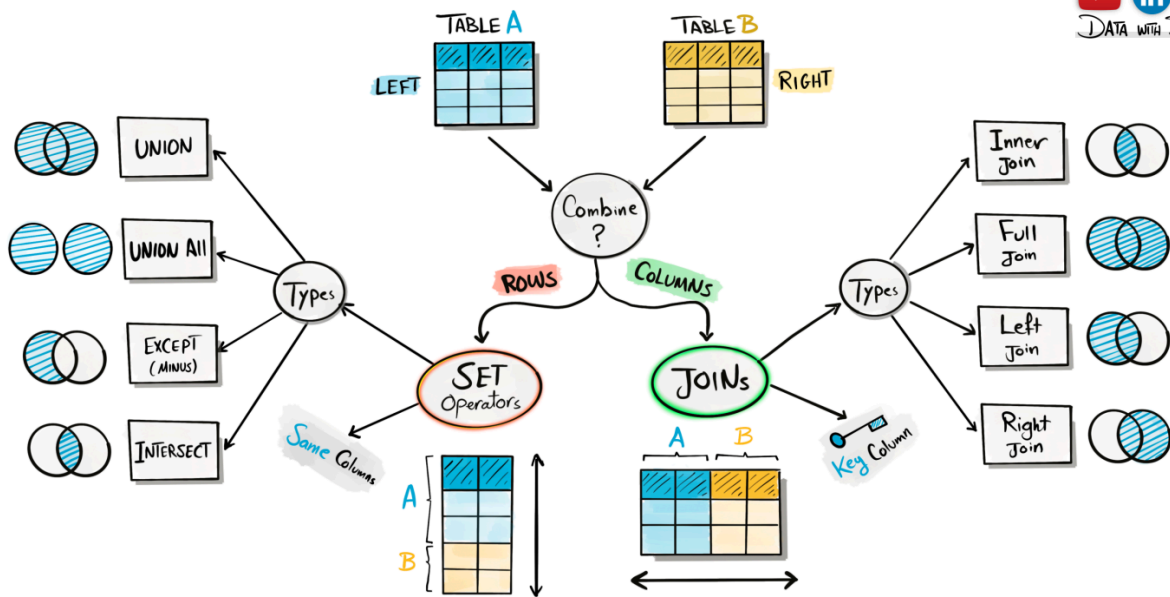


No alias → confusing / error



Interview One-Liner

“A self join is used when a table needs to be related to itself, typically for hierarchical or comparison-based queries.”



SET RULES

1. SET operators can be used in any clause.
2. ORDER BY is allowed only once—at the end of the query.
3. Each query must have the same number of columns.
4. Column data types must be compatible across queries.
5. The result set takes column names from the first query.

UNION ALL vs UNION

- **UNION ALL** is faster than **UNION** because it doesn't remove duplicates.
- Use **UNION ALL** if you are sure there are no duplicates.
- Use **UNION ALL** to identify duplicates and data quality issues

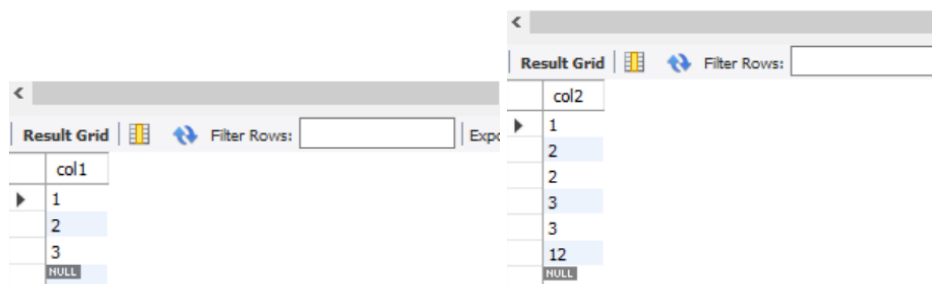
SET OPERATORS

UNION	UNION ALL	EXCEPT/MINUS	INTERSECT
Returns All rows from both sets, elimination duplicates	Returns All rows from both sets, including duplicates	Return unique rows in first set that are not in second table	Return only the common rows between two sets
<pre>SELECT FirstName, LastName FROM Customers UNION SELECT FirstName, LastName FROM Employees</pre>	<pre>SELECT FirstName, LastName FROM Customers UNION ALL SELECT FirstName, LastName FROM Employees</pre>	<pre>SELECT FirstName, LastName FROM Customers EXCEPT SELECT FirstName, LastName FROM Employees</pre>	<pre>SELECT FirstName, LastName FROM Customers INTERSECT SELECT FirstName, LastName FROM Employees</pre>

Both table1 and table2 have only one column so * works, else it needs to be specified-

```
1 • select * from table1;
2 • select * from table2;
```

```
1 • select * from table1;
2 • select * from table2;
```



col1
1
2
3
NULL

col2
1
2
3
12
NULL

```

1 • select * from table1
2   union
3   select * from table2;

```

col1
1
2
3
NULL
12

```

1 • select * from table1
2   union all
3   select * from table2;

```

```

1 • select col1 from table1
2   union
3   select name from employees;

```

col1
1
2
3
NULL
1
2
2
3
3
12
NULL

col1
1
2
3
NULL
Alice
Bob
Charlie
David
Eve
Frank
Grace

MySQL doesn't support EXCEPT and INTERSECT, so do like this-

i) For EXCEPT simulation (can use left anti join also)

```

1   -- MySQL doesn't support INTERSECT and EXCEPT SET operations
2 •  SELECT col1
3     FROM table1
4     WHERE col1 NOT IN (
5         SELECT col2 FROM table2
6     );

```

col1

⚠️ NULL Trap 😈

If `table2.col2` has NULL → this may return empty result

✅ Safer Option: NOT EXISTS (BEST)

```
</> SQL

SELECT col1
FROM table1 t1
WHERE NOT EXISTS (
  SELECT 1
  FROM table2 t2
  WHERE t2.col2 = t1.col1
);
```

👉 Preferred in interviews

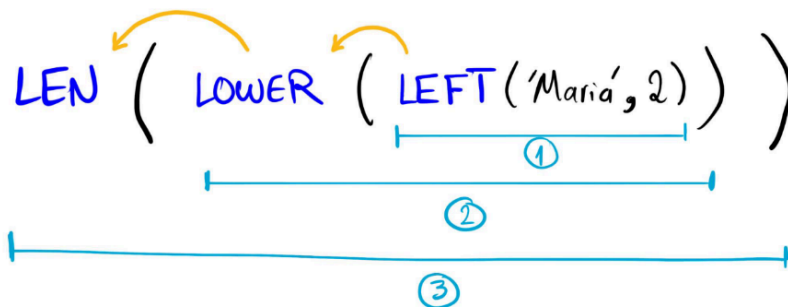
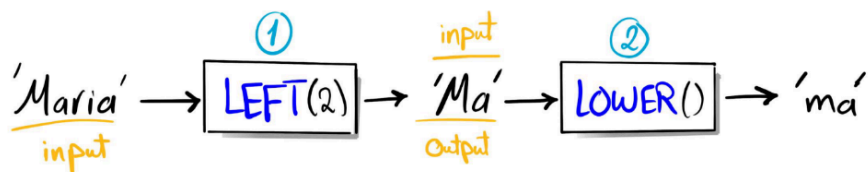


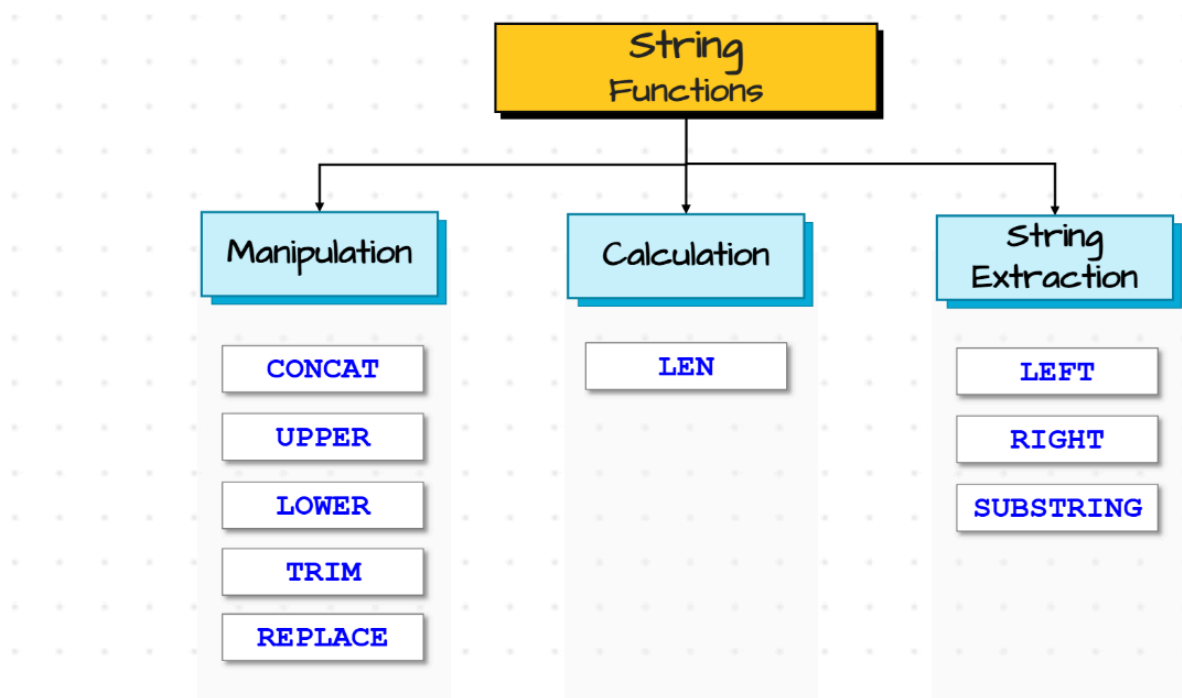
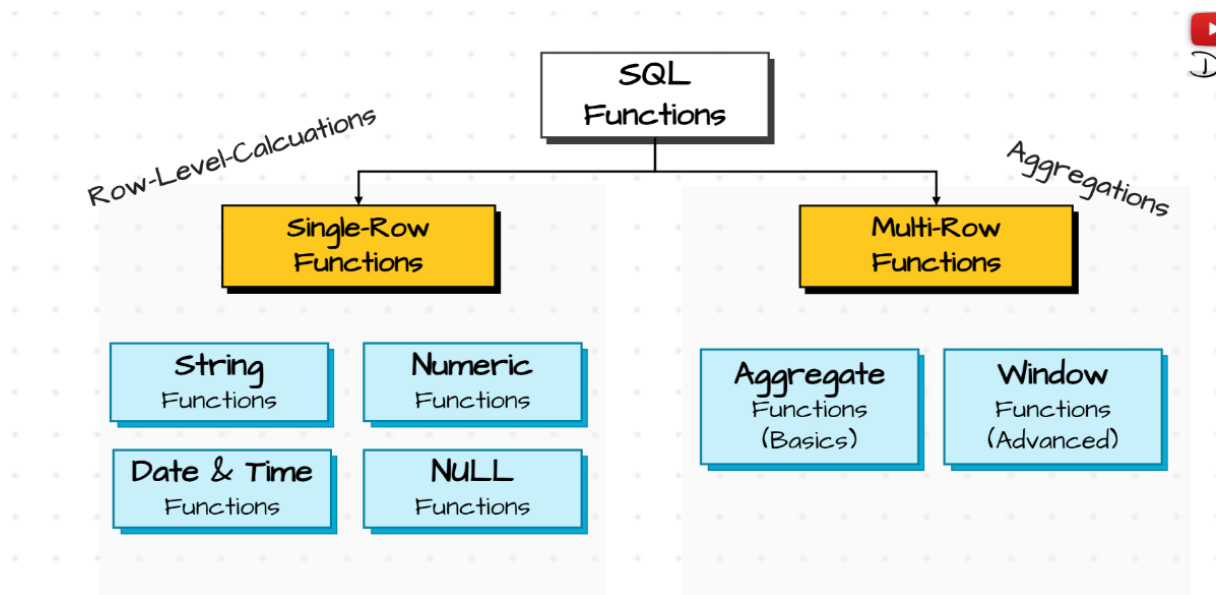
ii) For INTERSECT simulation-

Use INNER JOIN

FUNCTIONS-

Nested Function





In MySQL it's IFNULL() not ISNULL()-

ISNULL



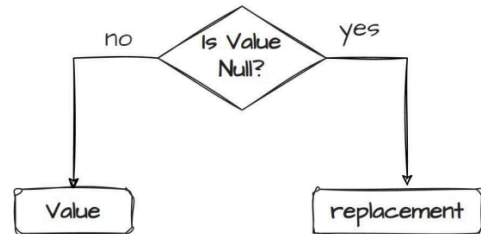
replaces NULL with the specified replacement value.

SYNTAX

```
ISNULL(value, replacement)
```

```
ISNULL(ShippingAddress, BillingAddress)
```

OrderID	Shipment Address	Billing Address	ISNULL
1	A	B	A
2	NULL	C	C
3	NULL	NULL	NULL



The `IFNULL()` function in MySQL is used to handle `NULL` values by replacing them with a specified fallback value. It is particularly useful for cleaning data in reports or preventing errors in mathematical calculations where a `NULL` would otherwise result in a `NULL` output. [DbVisualizer +4](#)

Syntax

sql



```
IFNULL(expression, alternate_value)
```

- **expression:** The value or column to test for `NULL`.
- **alternate_value:** The value to return if the first expression is `NULL`. [Scaler](#)

- **Handling Missing Table Data:** Provide a default value for a column.

sql



```
-- Displays 'No phone provided' for customers without a phone number
SELECT cust_code, IFNULL(phone_no, 'No phone provided') AS contact_info
FROM customer;
```

- **Mathematical Safety:** Prevent calculation errors.

sql



```
-- Ensures the calculation works even if 'Ordered' is NULL
SELECT product_name, price * (stock_amount + IFNULL(Ordered, 0))
FROM products;
```

The `COALESCE()` function in MySQL **returns the first non-null value in a list of expressions**. If all expressions in the list evaluate to `NULL`, the function returns `NULL`. [W3Schools +2](#)

Syntax

sql



```
COALESCE(value1, value2, ..., value_n)
```

- **Arguments:** The function accepts one or more expressions.
- **Data Types:** Arguments can be of any data type. If multiple data types are mixed, the function returns a value in the data type with the highest precedence. [GeeksforGeeks +3](#)

Common Use Cases

- **Providing Default Values:** Replace `NULL` with a more readable string or value (e.g., changing `NULL` to "N/A" or 0).

- **Math Operations:** Prevent `NULL` from breaking calculations by substituting it with 0 (e.g., `Price * (1 - COALESCE(discount, 0)/100)`).
- **Data Aggregation:** Use with aggregate functions like `SUM` or `MAX` to ensure a numeric result instead of `NULL` when no records exist. www.mysqltutorial.org +6

Key Comparisons

- **COALESCE vs. IFNULL:** While `IFNULL()` is specific to MySQL and accepts only **two** arguments, `COALESCE()` is an [ANSI-standard](#) function that can handle **any number** of arguments.
- **COALESCE vs. CASE:** The MySQL COALESCE function is effectively a shorthand for a [CASE expression](#).
 - `COALESCE(V1, V2)` is equivalent to `CASE WHEN V1 IS NOT NULL THEN V1 ELSE V2`
`END` . [GeeksforGeeks](#) +4